

CSE 144 Final Project Report Transfer Learning Challenge

Sam Hutchins (2167175)

Andrey Voronov(1759417)

Spring 2026

1 Introduction

1. The goal is to apply transfer learning techniques learned in the class to train a 100 class image classifier for a more difficult dataset. The resulting predictions are submitted to a kaggle competition for leaderboard scoring.

2. Transfer learning is the appropriate method for this challenge since there is a low number of images available to work with in the provided dataset. If you were to train a sophisticated neural network on a limited dataset of 1000 images, then you would immediately face issues of overfitting.

3. Our main approach involved understanding how to apply transfer learning with a number of fine-tuning techniques on a simple CNN first, Resnet50. We then took what we learned with this model and tried a more advanced model trained with self-supervised learning to allow for better generality due to its self-supervised nature. The DINOv3 ConvNeXt small was our choice to maintain benefits of a self-supervised architecture while avoiding issues of overfitting involved with larger models. We froze the backbone of the model while training the classifier head with a low learning rate.

2 Dataset

1. The dataset consisted of 1079 images, where we utilized a stratified split method of validation, splitting the dataset into 868 training images and 211 validation images.

2. The dataset directory is structured as follows:

train/

- 0/

- 0.jpg

- 1.jpg

...

- 9.jpg

1

- 1/

...

- 99/

...

test/

- 0.jpg

- 1.jpg

...

- 999.jpg

- sample_submission.csv. Due to torchvision.datasets.ImageFolder's sorting method, the folder names were sorted lexicographically as strings, so folder "10" receives the index 2, instead of 10. If the predictions were submitted using these indices, the label order would appear random, assessing the classifications at random. To avoid this issue we created a dictionary to invert class_to_idx into idx_to_class and reverted each prediction when assembling predictions back into the required submission.csv file.

3. Preprocessing and augmentation:

- Training: RandomResizedCrop(224), RandomHorizontalFlip, conversion to tensors, and normalization (mean [0.485, 0.456, 0.406], std [0.229, 0.224, 0.225]), matching the backbone's pretraining statistics
- Validation/test: deterministic Resize(256) + CenterCrop(224) with the same normalization.
- MixUp (alpha = 0.5) applied per batch during training with torchvision.transforms.v2.MixUp, which blends pairs of images and their one-hot labels.

3 Implementation

3.1 Model

1. We used the convnext_small.dinov3_lvd1689m model from the timm library (after consulting the timm leaderboard). It is a ConvNeXt-Small CNN where the weights come from DINOv3 self-supervised distillation on the LVD-1689M dataset. We chose this model for the benefit of self-supervised learning being

great with smaller datasets due to its implicit generality that comes with self-supervised learning. They are trained to be informative without label supervision, so a simple linear classifier on top of them is self sufficient. We also anticipate the small version to be light enough to train quickly on our GPU, and work well with 224x224 inputs recommendation as opposed to the high resolution requirement of ViT models.

2. The model's classifier is modified from 1000 classes down to `num_classes=100`, which replaced the linear classifier, while ensuring the weights are unfrozen for the training step. We did not want to risk unfreezing the backbone, avoiding the issue of clobbering earlier weights the model worked hard to train.

3. We overrode the model's `train()` function to force the frozen backbone into `eval()` mode while the linear classifier layer trains. We figured out that we needed this following the Resnet50 test run, as the batch normalization statistics were being calculated over each train iteration if we did a simple `requires_grad=False` on the backbone. Eval mode ensured the backbone would not manipulate batch normalization statistics.

1

3.2 Training

1. The loss function we used was cross-entropy (`nn.CrossEntropyLoss`), and the optimizer we used was AdamW overhead parameters only.

2. We chose to use the CosineAnnealing scheduler, learning rate= $1e-3$, weight decay= $1e-2$, batch size=64, epochs=25

3. AMD Radeon RX 7900 XT (gfx1100) via ROCm-enabled PyTorch on NixOS
torch: 2.11.0 timm: 1.0.26

4 Experiments

1. The first model was partially fine-tuned ResNet50 (ImageNet supervised weights, layer 4 and head unfrozen). It reached roughly 0.57-0.66 validation accuracy across runs with 25 epochs and a batch size of 128. The kaggle score was around 59-62%. The supervised ImageNet features were found to be weaker than the modern self-supervised features for transfer learning over a small dataset. Overfitting was a constant issue when trying to tune for increased performance.

2. We tried to change our validation method from our simple stratified split to a k-fold validation method after setting up the DINOv3 model, but had issues implementing it, leading to a lower accuracy of 0.77%.

3. Ablations

- Freezing strategy: frozen backbone + trainable head consistently beat partial fine-tuning at this data scale with ConvNeXt small. Unfreezing more

layers increased train accuracy but hurt validation. This is a sign of overfitting.

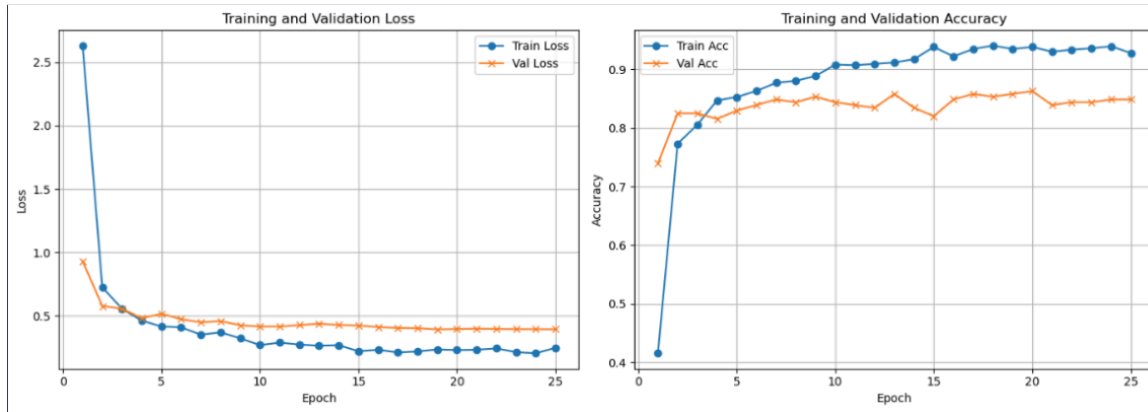
- Backbone family: self-supervised pretraining outperformed supervised ImageNet weights (ResNet50) as the frozen backbone.
- Ensembling: We tried implementing a 3-backbone ensemble (DINOv3 ViT + EVA02 + DINOv3 ConvNeXt, to try to account for each others blind spots found in their training). The training took a while and with our setup only improved validation slightly on test, so we kept the simple design of a single DINOv3 ConvNeXt small.

5 Results

1. Training/validation accuracy (and loss) and any key curves/plots.



This graph is from the ResNet50 where a noticeably large gap between train acc and val acc shows overfitting issues.



Our final Training/Validation Loss and Training/Validation accuracy graphs. Using the DINOv3 ConvNeXt small.

2. Kaggle public leaderboard.

17	Kappa Chungus De luxe	  	0.87272	2	2d
18	Suchana Poudel		0.87272	3	20h
19	Andrey Voronov	 	0.86363	2	19h
 Your Best Entry! Your most recent submission scored 0.86363, which is an improvement over your previous score of 0.57272. Great job! Tweet this					
20	Ben Liu	  	0.85454	5	10d
21	Skye Gunasekaran		0.85454	1	8d

6 Discussion

1. What worked best was a self-supervised backbone, kept entirely frozen, with a single 100 head trained under heavy regularization (MixUp + weight decay +

cosine annealing). At around 10 images per class, the main issue was balancing between a strong model with efficient training that does not overfit to the dataset.

2. We chose the single frozen ConvNeXt small to ensure we got the benefits of a self supervised learning model while avoiding heavy weight models that would quickly overfit. The two main issues we found while experimenting with the ResNet50 model was first an issue where train accuracy was oscillating due to learning rate issues, where after fine-tuning the learning rate we saw a drop in oscillation. We also faced an issue where batch normalization was changing the accuracy of our model silently until we ensured the model's backbone that was supposed to be frozen was set to eval mode.

2. Additional fine-tuning could increase our model accuracy if we had more time. If we also were able to get the ensemble strategy working with 3 models, we could improve our accuracy even further. Another improvement could be figuring out how to apply pseudo labeling, effectively increasing the number of images in our dataset that we can work with, while maintaining any invariants present with the dataset. In addition, exploring an entirely new model in order to attempt transfer learning could get us closer to the ideal goal of 99+% accuracy compared to the current 86%.

7 Reproducibility

Seed and determinism:

- `set_seed(42)` seeds Python, NumPy, and PyTorch.
- `cudaNN.benchmark=False`, `cudaNN.deterministic=True`, and `torch.use_deterministic_algorithms(True)` are enabled
- DataLoaders use `num_workers=0` so sample ordering is identical across platforms

Environment: Nix flake will handle all versioning if using the nix package manager. otherwise: Python: 3.13.12, PyTorch: 2.11.0, timm: 1.0.26, torchvision, kagglehub, matplotlib

Data: downloaded via `kagglehub.competition_download("ucsc-cse-144-spring-2026-final-project")` with Kaggle API credentials in `~/.kaggle/kaggle.json`.

To train: run CSE-144_FINAL.ipynb top to bottom. The training loop checkpoints the `best_convnext_small.pt` accuracy to `./checkpoints/`.

To generate the submission: the final notebook section loads the best checkpoint, runs inference on test/, decodes indices through `idx_to_class`, and writes `submission.csv` with columns {ID, Label}.

Best model checkpoint:

https://drive.google.com/file/d/1BiEXp_Nmf7-2lxfw2U4wJou2gtCy_gl6/view?usp=sharing

8 Team Contributions

1. Samuel Hutchins: model architecture design, report
2. Andrey Voronov: fine-tuning, report

9 References

1. TorchVision documentation / pretrained model sources.

2. Any papers or tutorials used.

<https://huggingface.co/timm>

<https://ai.meta.com/research/dinov3/>

<https://arxiv.org/abs/2508.10104>

<https://www.lightly.ai/blog/dinov3>

<https://towardsdatascience.com/getting-started-with-pytorch-image-models-timm-a-practitioners-guide-4e77b4bf9055-2/>

<https://timm.fast.ai/>

<https://docs.pytorch.org/docs/2.12/data.html>

<https://docs.python.org/3/reference/datamodel.html>

<https://docs.pytorch.org/docs/2.12/accelerator.html>

<https://docs.pytorch.org/docs/2.12/tensors.html>

https://docs.pytorch.org/docs/2.12/generated/torch.optim.lr_scheduler.CosineAnnealingLR.html

<https://docs.pytorch.org/docs/2.12/generated/torch.optim.AdamW.html>